

Software Architecture and Design Specification

for

Vehicle Service & Maintenance Management System

Document Item	Details
Version	1.0
Status	Final Submission
Date	06-Oct-2026
Course	Software Engineering

Prepared by

Priya K L - PES1UG24CS600

Prajwal S J - PES1UG24CS594

Harshith S - PES1UG24CS574

Basavaraj Mruthyunjaya Mudakammanavar - PES1UG24CS561

Revision History

Name	Date	Reason for Changes	Version
Project Team	01-Oct-2026	Initial SRS baseline used for architecture and design	1.0
Project Team	06-Oct-2026	SAD prepared for final submission	1.0

Table of Contents

1. Introduction.....	1
1.1 Purpose	1
1.2 Scope	1
1.3 Audience	1
1.4 Definitions	1
2. Document Overview	2
2.1 How to Use This Document	2
2.2 Related Documents.....	2
3. Architecture.....	3
3.1 Architecture Goals and Constraints	3
3.2 Stakeholders and Concerns	3
3.3 Component Diagram	3
3.4 Component Descriptions	4
3.5 Chosen Architecture Pattern and Rationale	4
3.6 Technology Stack & Data Stores	4
3.7 Risks & Mitigations	5
3.8 Traceability to Requirements.....	5
3.9 Security Architecture	5
4. Design	6
4.1 Design Overview.....	6
4.2 UML Sequence Diagrams	6
4.2.1 Create, Review and Validate Service Request	6
4.2.2 Assign Work, Perform Service and Complete Request	7
4.3 API Design	8
4.4 Error Handling, Logging & Monitoring	8
4.5 UX Design	9
4.6 Open Issues & Next Steps	9
5. Appendices	10
5.1 Glossary	10
5.2 References	10
5.3 Tools	11

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification (SAD) defines the architecture and detailed design of the Vehicle Service & Maintenance Management System (VSMMS). It translates the approved Software Requirements Specification (SRS) into an implementable architecture, component structure, system interactions, interfaces, security controls and design decisions.

1.2 Scope

The document covers release 1.0 architecture and design for customer and vehicle registration, service-request intake and validation, service work assignment, service execution and recording, service status and history, search, and user/access administration.

1.3 Audience

Developers, architects, testers, project managers, UI/UX designers, service-centre stakeholders and course evaluators.

1.4 Definitions

Definitions follow the SRS glossary, including Functional Requirement (FR), Non-Functional Requirement (NFR), Application Programming Interface (API), Role-Based Access Control (RBAC), Representational State Transfer (REST), Unified Modeling Language (UML), and Vehicle Service & Maintenance Management System (VSMMS).

2. Document Overview

2.1 How to Use This Document

Section 3 describes the architecture, components, architecture pattern, technology decisions, risks, traceability and security architecture. Section 4 describes detailed design, sequence diagrams, API interfaces, error handling, logging, monitoring and user experience. Section 5 provides supporting glossary, references and tools.

2.2 Related Documents

Document	Purpose
Software Requirements Specification (SRS), Version 1.0	Defines functional/non-functional requirements, analysis models, field layouts and traceability.
Software Test Plan	Defines testing approach, security validation, test cases and requirement traceability.
UML Analysis Models	Provides the use-case, activity and entity-relationship models used as inputs to design.

3. Architecture

3.1 Architecture Goals and Constraints

- Separate presentation, application/business and data responsibilities.
- Provide modular components that can be developed and tested independently.
- Support the complete service-centre workflow.
- Enforce role-based access and protect customer/service data.
- Maintain data consistency and integrity.
- Support the SRS performance target.

Key constraints from the SRS:

- Browser-based application.
- HTTPS with TLS 1.2 or higher.
- Relational database with transaction support.
- Modular and separately testable design.
- English-only release 1.0 with texts externalised for future translation.
- Small student team and one academic term.

3.2 Stakeholders and Concerns

Stakeholder	Main Concern
Customer / Vehicle Owner	Easy requests and visibility of status/history.
Service Advisor / Service Staff	Fast registration, review, validation, assignment and status management.
Service Technician	Clear assigned work and reliable service recording.
Administrator	Secure user, role and configuration management.
Development Team	Modularity, maintainability, testability and clear interfaces.
Test / QA Team	Traceability and verifiable system behaviour.

3.3 Component Diagram

The component diagram uses the logical components defined in SRS Section 2.1.

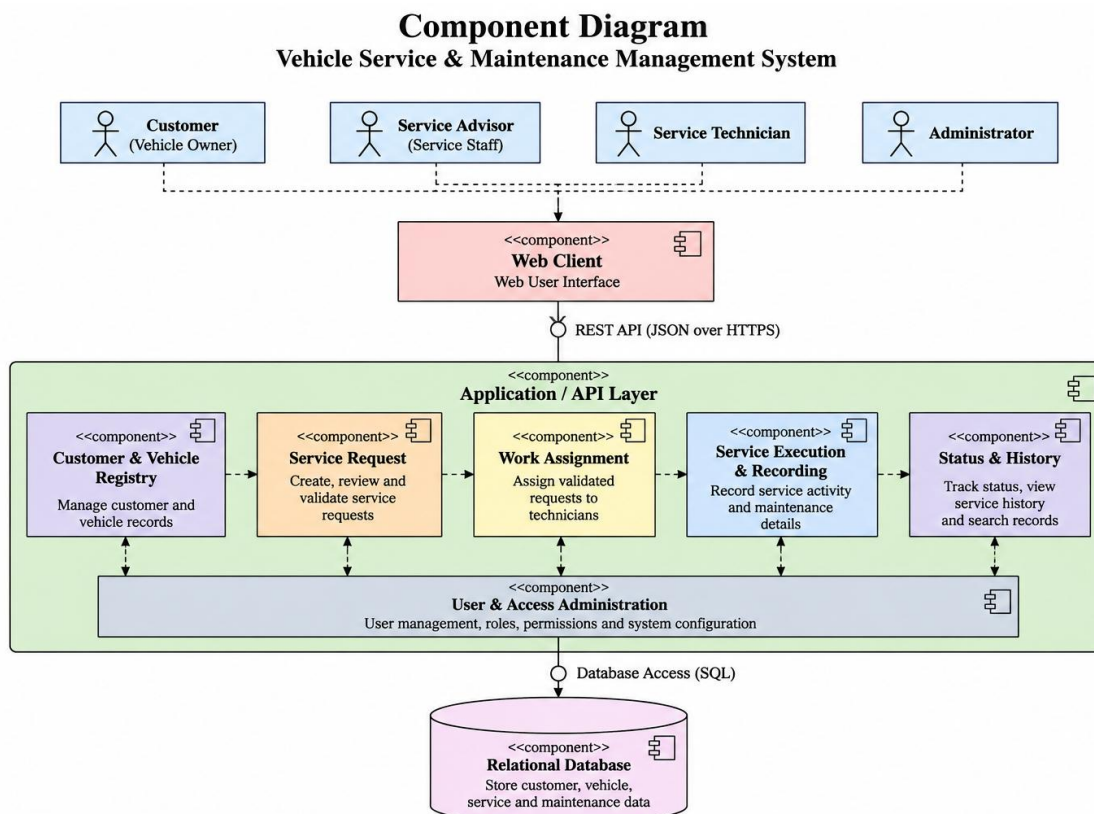


Figure 3-1: UML-style Component Diagram

The Web Client is used by all system users. It communicates with the application through the REST-style API. Application components implement business functions and access the relational database through database access services.

3.4 Component Descriptions

Component	Description	Requirement
Web Client	Responsive browser interface for all users.	FR-001 to FR-006
Customer & Vehicle Registry	Registers and maintains customer and vehicle records.	FR-001
Service Request	Creates, reviews and validates service requests.	FR-002
Work Assignment	Assigns validated requests to technicians and supports assigned-work viewing.	FR-003
Service Execution & Recording	Records service activity, maintenance details and completion information.	FR-004
Status & History	Tracks status, service history and record search.	FR-005
User & Access Administration	Manages users, roles, permissions and system configuration.	FR-006
Relational Database	Persistent store for customers, vehicles, requests, activities, maintenance records and users.	FR-001 to FR-006

3.5 Chosen Architecture Pattern and Rationale

The chosen architecture pattern for the design is Layered Architecture. This is a design decision for the SAD; the SRS does not explicitly name the pattern.

Layer	Responsibility
Presentation Layer	Responsive Web Client and user interaction.
Application / Business Layer	Validation, assignment, service execution, status/history and access-control logic.
Data Access Layer	Controlled database operations, parameterised SQL and transaction handling.
Data Layer	Relational database for persistent records.

The layered approach is suitable because it separates responsibilities and supports maintainability, testability and the SRS requirement for modular, separately testable modules.

3.6 Technology Stack & Data Stores

Area	Decision / Status
Client	Responsive web application
Communication	REST-style API
Data format	JSON / UTF-8
Web security	HTTPS using TLS 1.2 or higher
Server	Linux-based server or container platform
Database	TBD — database product and implementation not finalized
Database access	TBD — database access method not finalized
Password hashing	bcrypt or Argon2
Frontend framework	TBD — exact framework not specified in SRS
Backend framework	TBD — exact framework not specified in SRS
Database product/version	TBD — SRS permits PostgreSQL or MySQL 8+

The exact database product and database access method are not finalized at this stage. They will be updated after the team decides the implementation technology.

3.7 Risks & Mitigations

Risk	Impact	Mitigation
Unauthorized access	Data exposure or modification	Authentication, RBAC and authorization checks.
Invalid request	Incorrect work assignment	Validate mandatory fields and vehicle registration.

Database failure	Loss of availability	Automated daily backups and controlled database operations.
Incorrect status	Incorrect information	Validate status transitions and timestamp changes.
Performance degradation	Workshop delays	Performance testing against the 50-user target.
Post-completion modification	Record integrity loss	Make completed entries read-only.
Incorrect role assignment	Unauthorized functions	Restrict role management to administrators.

3.8 Traceability to Requirements

Requirement	Architecture Component	Design Reference
FR-001	Customer & Vehicle Registry	Sequence 1; customer/vehicle design
FR-002	Service Request	Sequence 1; validation design
FR-003	Work Assignment	Sequence 2; assignment design
FR-004	Service Execution & Recording	Sequence 2; service recording design
FR-005	Status & History	Sequence 2; status/history design
FR-006	User & Access Administration	Security architecture; access-control design
NFR-001	Web Client + Application + Database	Performance design/monitoring
NFR-002	User & Access + Security + Database	Security architecture

These references are intended to replace the architecture/design TBD entries in the SRS Requirement Traceability Matrix during final document integration.

3.9 Security Architecture

Security Control	Design
Authentication	Users authenticate before protected functions are accessed.
Role-Based Access Control	Permissions are based on Customer, Service Advisor, Service Technician or Administrator role.
Authorization	Protected operations are checked against the authenticated user's role.
Transport Security	Browser-server communication uses HTTPS with TLS 1.2 or higher.
Password Protection	Passwords are stored only as salted hashes using bcrypt or Argon2.
Audit Logging	Status changes and record edits are timestamped and attributed to the acting user.
Data Integrity	Completed service activity and maintenance records are read-only.

4. Design

4.1 Design Overview

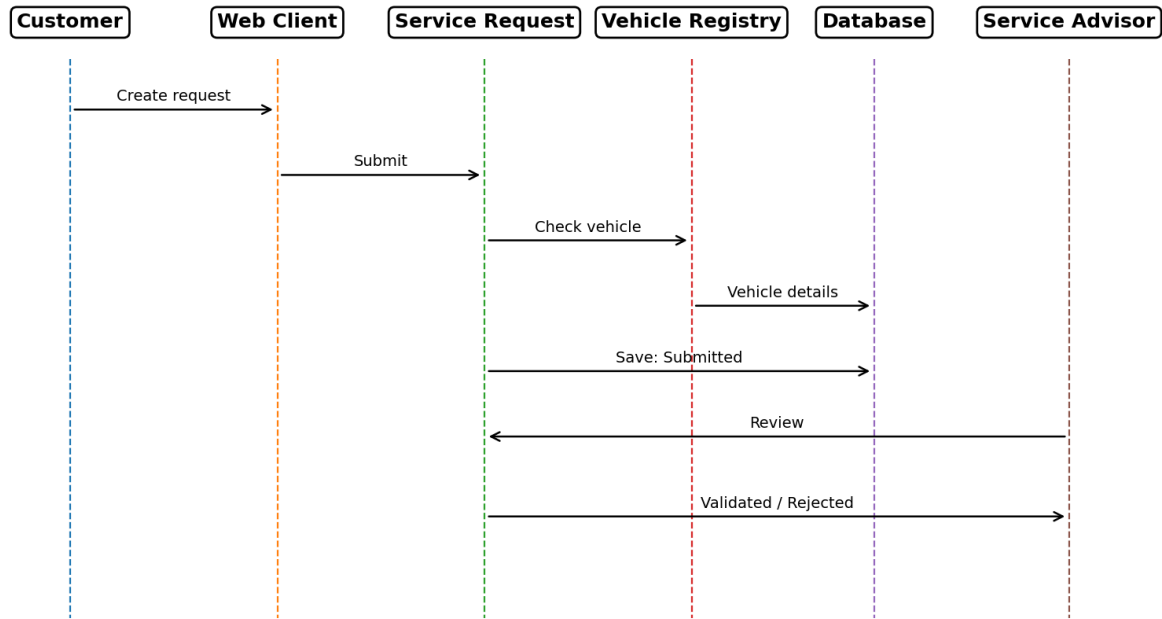
The detailed design translates the architecture into implementable interactions. The Web Client communicates with application components through a REST-style API using JSON. Application components enforce business rules and role permissions, while the data access layer performs controlled operations on the relational database.

- Create, review and validate a service request.
- Assign a validated request to a Service Technician.
- Perform service and record maintenance details.
- Complete a service request and update status/history.
- Control access according to user roles.

4.2 UML Sequence Diagrams

4.2.1 Create, Review and Validate Service Request

This sequence represents the FR-002 workflow from request creation through validation or rejection.

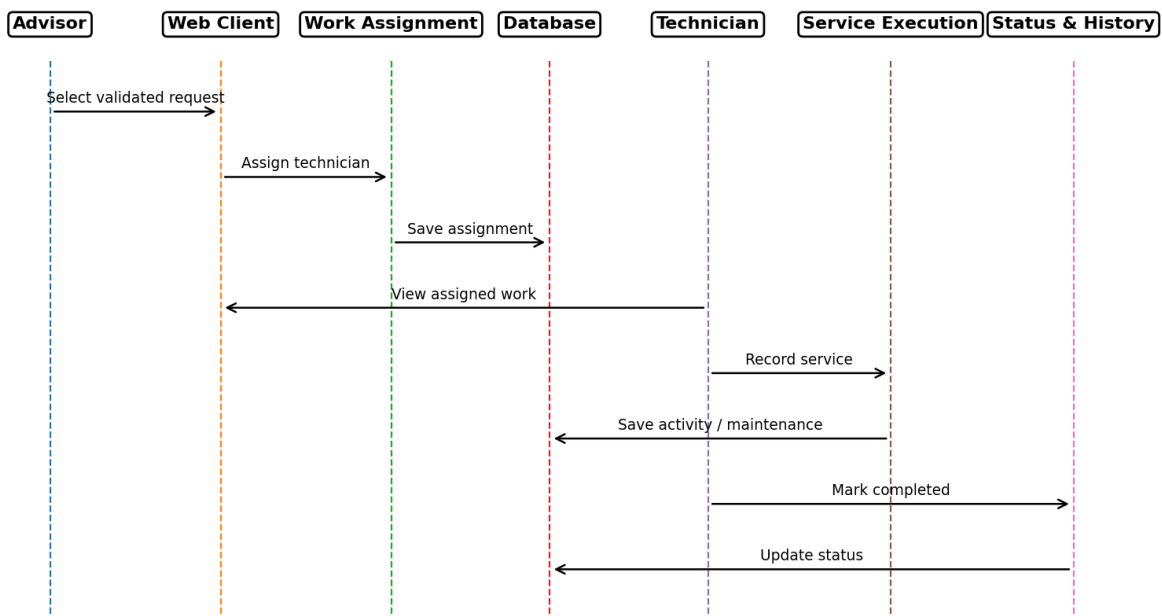


Sequence 1 — Create, Review and Validate Service Request (FR-002)

Figure 4-1: Sequence Diagram — Create, Review and Validate Service Request

4.2.2 Assign Work, Perform Service and Complete Request

This sequence covers FR-003 and FR-004, from assignment of a validated request through service execution and completion.



Sequence 2 — Assign Work, Perform Service and Complete Request (FR-003, FR-004)

Figure 4-2: Sequence Diagram — Assign Work, Perform Service and Complete Request

4.3 API Design

The SRS specifies a REST-style API using JSON. The following endpoint definitions are proposed for implementation; exact route naming may be finalized by the development team.

Method	Endpoint	Purpose
POST	/api/service-requests	Create service request.
GET	/api/service-requests/{id}	Retrieve service request.
PUT	/api/service-requests/{id}/status	Update status.
POST	/api/service-requests/{id}/assign	Assign technician.
POST	/api/service-requests/{id}/activities	Record service activity.
GET	/api/vehicles/{id}/history	Retrieve vehicle service history.

Example request:

```
{ "vehicleId": "V001", "serviceType": "Oil Change", "description": "Engine oil replacement" }
```

Example response:

```
{ "requestId": "SR001", "status": "Submitted" }
```

4.4 Error Handling, Logging & Monitoring

Condition	Expected Handling
Invalid login	Reject authentication and display a general authentication error.
Missing mandatory field	Display a field-level validation message.
Vehicle not registered	Reject request and explain validation failure.
Unauthorized operation	Reject operation and return access-denied response.
Request not found	Return a not-found response and clear message.
Database failure	Log technical error and display a general failure message.
Invalid status transition	Reject operation and preserve current valid status.

Logging should cover authentication/security events, status changes, record edits, administrative changes and application/database errors. Monitoring should cover application errors, API response time, database availability and performance-test results.

4.5 UX Design

The SRS requires a responsive interface from 360 px width, consistent Save/Cancel/Back/Help controls, inline error messages, keyboard navigation, sufficient colour contrast, text alternatives and WCAG 2.1 Level AA accessibility.

Screen	Users	Purpose
Sign-up / Log-in	All	Authentication.
Customer & Vehicle Registration	Service Advisor	Register/maintain records.
My Vehicles	Customer	View vehicles.
Create Service Request	Customer, Advisor	Submit request.
Review Requests	Advisor	Review/validate/reject.
Assign Service Work	Advisor	Assign validated work.
My Assigned Work	Technician	View assigned requests.
Service Execution / Recording	Technician	Record service/maintenance.
Service Status & History	Customer, Advisor	View status/history.
Search Records	Advisor, Administrator	Search records.
User & Access Management	Administrator	Manage users/roles/settings.

4.6 Open Issues & Next Steps

- Finalize exact frontend framework/version.
- Finalize exact backend framework/version.

- Finalize relational database product/version.
- Finalize API endpoint naming.
- Finalize deployment/hosting configuration.
- Complete final UI screen designs.
- Update SRS RTM with final architecture/design references.
- Execute tests and record Actual Result and Pass/Fail in the Test Plan.

5. Appendices

5.1 Glossary

Term	Definition
API	Application Programming Interface.
FR	Functional Requirement.
NFR	Non-Functional Requirement.
RBAC	Role-Based Access Control.
REST	Representational State Transfer.
JSON	JavaScript Object Notation.
SQL	Structured Query Language.
TLS	Transport Layer Security.
UML	Unified Modeling Language.
UX	User Experience.
SAD	Software Architecture and Design Specification.
SRS	Software Requirements Specification.
VSMMS	Vehicle Service & Maintenance Management System.
WCAG	Web Content Accessibility Guidelines.

5.2 References

1. PES University, Department of Computer Science and Engineering, Software Requirements Specification for Vehicle Service & Maintenance Management System, Version 1.0, 01-Oct-2026.
2. PES University, Software Architecture and Design Specification template/sample, Software Engineering course material.
3. PES University, Software Test Plan and test cases, Software Engineering course material.
4. ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering.
5. IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
6. Object Management Group, Unified Modeling Language (UML) Specification, Version 2.5.1.
7. OWASP Application Security Verification Standard (ASVS), Version 4.0.3.
8. Digital Personal Data Protection Act, 2023.

5.3 Tools

Tool / Technology	Purpose
GitHub	Version control and team collaboration.
Microsoft Word	SAD preparation and final submission.
UML diagramming tool	Component and sequence diagrams.
Visual Studio Code	Software development and source-code editing.
Relational database tool	Database development and testing.

End of Software Architecture and Design Specification